RTL for AXI4 (Master & Slave) to PCIe 6 Link

SARCathon 2026 – Semiconductor Challenge

Stage 1 Qualifier – Technical Design Report

Design under test	`axi4_to_pcie_bridge` (single Verilog-2001 module)
Testbench	Self-checking directed testbench, 10 scenarios, 69 checks
Simulator	Icarus Verilog 12.0
Result	**69 / 69 PASS, 0 FAIL**

Team Details

Team Name: The Chipmakers

Name	Role	Contact
Shivam Goyal	Team Leader	+91 94170 52110
Aman Devgan	Team Member	+91 86850 03501
Lipika Lipika	Team Member	+91 97800 22724

All team members verified.

---

1 Problem Understanding and Selected Implementation Scope

The SARCathon 2026 Semiconductor Challenge asks teams to build a hardware bridge between an AXI4/AXI4-Stream SoC-side interface and a PCIe 6 host-facing interface, following the four-layer reference architecture supplied with the problem statement: (1) an AXI4-Stream interface, (2) an interconnect layer with a central crossbar and dynamic arbitration, (3) a link layer that packetizes data into fixed 256-byte Flow Information Units with FEC/CRC, and (4) a protocol-to-PHY layer that performs multi-lane striping and PAM4 symbol mapping toward the analog SerDes. Stage 1 explicitly does not require a complete implementation; it rewards a smaller, correctly designed, and thoroughly verified subset over a large but unverified one.

Given the one-week Stage 1 timeline, the team selected the following implementation scope:

- A synthesizable **AXI4 (memory-mapped) slave-side bridge**, `axi4_to_pcie_bridge`, that accepts AXI4 write and read transactions and converts each AXI beat into an abstract PCIe Transaction-Layer Packet (TLP) request, and converts PCIe completions back into AXI4 read/write responses.
- This corresponds functionally to the **transaction-level portion of the Interconnect Layer and the request/response half of the Link Layer** in the reference architecture: it is the logic that would sit between the AXI4 ingress/egress and the packetizer, deciding what gets sent as a PCIe request and how a completion is folded back onto the AXI bus.

- The AXI4-Stream 512-bit datapath, the central crossbar/arbiter for multiple stream masters, the 256-byte FEC/CRC packetizer, and the multi-lane PAM4 PHY-facing layer are treated as **out of Stage-1 scope** and are described only at the architecture level in Section 3. The abstract TLP interface used here is the intended attachment point for that future work.

This scope was chosen because it is the piece of the architecture that carries the actual AXI4 Master/Slave semantics named in the problem title, it is fully synthesizable RTL, and it can be exhaustively self-checked in simulation without depending on an unimplemented PHY or SerDes model, consistent with the Stage-1 evaluation philosophy of "a smaller but correctly designed and thoroughly verified implementation."

2 Background and Protocol Theory

This section summarizes the two protocols the bridge translates between, and the target use cases, to motivate the design decisions taken in Section 3.

2.1 Why Bridge AXI4 to PCIe?

On-chip SoC and accelerator fabrics almost universally use AXI4 (memory-mapped) or AXI4-Stream internally, because it is the de-facto standard for on-die interconnect in FPGA and ASIC design flows. Getting data off-chip to a host CPU, however, requires PCI Express, which is a packet-switched, serial, credit-based link protocol with a completely different transaction model (split request/completion, TLP framing, lane-based physical signaling). A bridge is therefore required at the boundary that (a) understands AXI4 transaction semantics well enough to generate correct AXI responses, and (b) understands enough of the PCIe transaction layer to generate well-formed requests and consume completions. Figure 1 illustrates the two motivating use cases from the problem statement – SmartNIC packet processing and ML inference acceleration – both of which reduce to the same shape: a streaming, high-bandwidth producer that must land data directly into host memory over PCIe with minimal software in the loop.

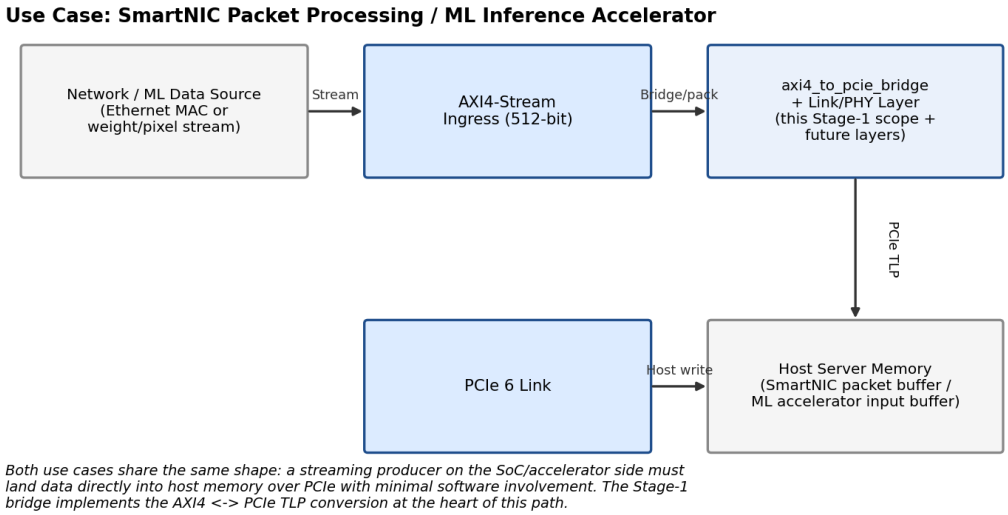


Figure 1: Both target use cases place the AXI4-to-PCIe bridge directly on the data path between an on-chip stream producer and host server memory.

2.2 AXI4 Protocol Fundamentals

AXI4 (Advanced eXtensible Interface, part of ARM's AMBA family) is a point-to-point, memory-mapped protocol built from **five independent channels**, shown in Figure 2: Write Address (AW), Write Data (W), Write Response (B), Read Address (AR), and Read Data (R). Each channel carries its own `VALID/READY` handshake pair and transfers exactly one item per cycle in which both signals are high (Figure 3). Three protocol properties are directly relevant to the bridge design:

- **Channel independence.** AW and W are logically separate channels and are not required to arrive in the same cycle – a compliant slave must be able to accept them in either order or interleaved. The bridge implements this with independent `aw_hold` / `w_hold` capture registers (Section 3, Test 3).
- **VALID stability.** Once a source asserts `VALID`, the protocol requires the associated payload to remain stable and `VALID` to remain high until `READY` is observed high – `VALID` must never wait for `READY` (no combinational `VALID`-on-`READY` loops). The bridge reuses this same discipline on its internal abstract PCIe TX interface (Test 6).
- **Burst semantics.** A single AW/AR transaction can describe a burst of up to 256 beats (`AxLEN`), with `AxBURST` selecting FIXED (same address every beat) or INCR (address increments by the transfer size each beat) addressing, and `WLAST` / `RLAST` marking the final beat of a write/read burst respectively.

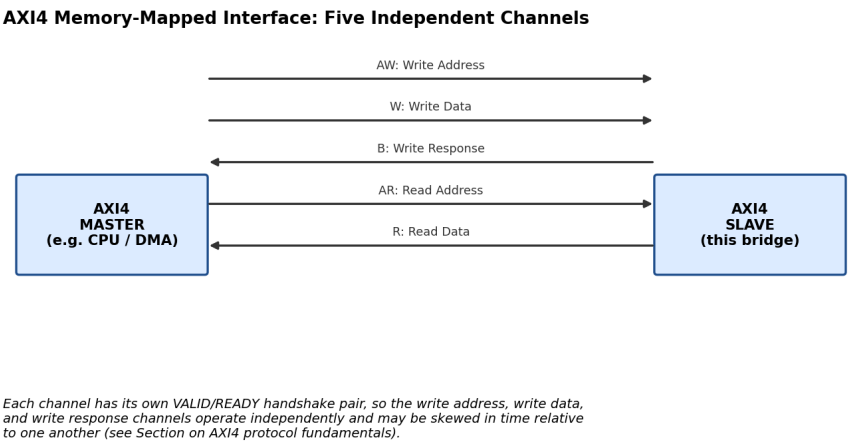


Figure 2: The five AXI4 channels between a master and this bridge (acting as an AXI4 slave).

2.3 PCIe Transaction-Layer Fundamentals

PCI Express organizes traffic into **Transaction Layer Packets (TLPs)**. For a memory-mapped bridge, the two request types that matter are **Memory Write (MemWr)**, a posted request that carries an address and a data payload with no completion expected, and **Memory Read (MemRd)**, a non-posted request that carries only an address/length and is answered later by a **Completion with Data (CplD)** carrying a tag that matches the original request. Every non-posted request carries a **tag** used to associate a later completion with the request that caused it, which is essential when multiple reads may be in flight. Figure 4 shows the conceptual fields the abstract Stage-1 TX/RX interface models; a full PCIe 6.0 implementation would additionally frame these fields into a FLIT-based, PAM4-encoded serial stream, which is the role of the Link and Protocol-to-PHY layers described in Section 3 and left for Stage 2.

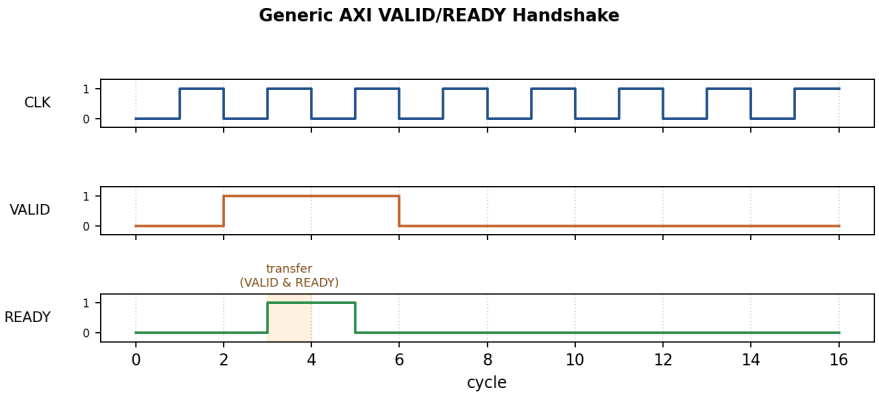


Figure 3: Generic AXI VALID/READY handshake: a transfer occurs on any cycle where both signals are sampled high at the clock edge.

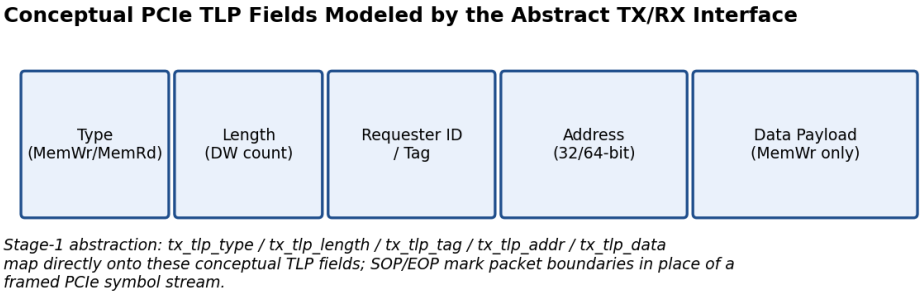


Figure 4: Conceptual PCIe TLP fields, and how they map onto the abstract `tx_tlp_*` / `rx_cpl_*` signals implemented in Stage 1.

2.4 Bridging Model

Given the above, the bridge's job reduces to a well-defined mapping: each AXI write beat becomes one MemWr TLP (posted, no completion needed, so the AXI `BRESP` can be returned as soon as the TLP is accepted downstream); each AXI read beat becomes one MemRd TLP that must wait for a tag-matched CplD before the corresponding AXI `RDATA`/`RRESP` can be returned. This one-beat-to-one-TLP mapping is a deliberate Stage-1 simplification – a production design would coalesce contiguous beats into wider TLPs to reduce PCIe overhead – and is called out again in Section 8.

3 Reference Architecture and Block-Level Design

Figure 5 reproduces the four-layer reference architecture from the problem statement and marks the portion carried into Stage 1. Figure 6 shows the Stage-1 block diagram of the implemented module and its external ports.

3.1 Module Interfaces

AXI4 Slave Port. Standard AXI4 write address (AW), write data (W), write response (B), read address (AR), and read data (R) channels, parameterized by `DATA_WIDTH` (default 64), `ADDR_WIDTH` (default 32) and `ID_WIDTH` (default 4). AW and W are captured independently (they need not arrive in the same cycle), matching the AXI4 protocol requirement that address and data channels are logically independent.

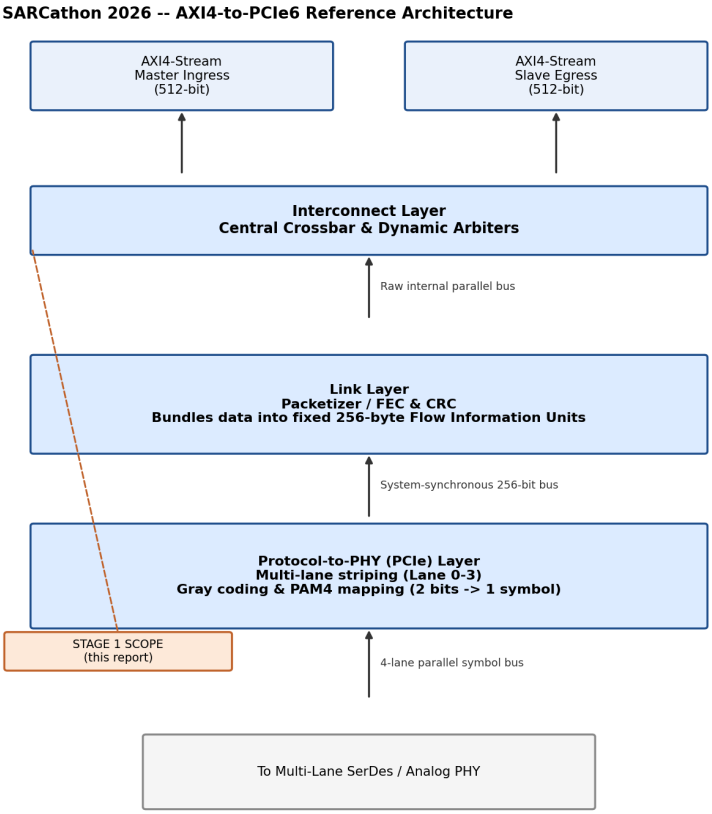


Figure 5: Full reference architecture (AXI4-Stream → Interconnect → Link Layer → Protocol-to-PHY). The dashed line marks the Stage-1 implementation scope.

Abstract PCIe TX TLP interface. A valid/ready request channel carrying TLP type (MemWr/MemRd), address, length (in DW), tag, payload, and start/end-of-packet markers. This is a Stage-1 abstraction of what the Link Layer's packetizer would consume; it is not a physical PCIe PIPE/PHY interface, consistent with the problem statement's own description of the reference design.

Abstract PCIe RX Completion interface. A valid/ready channel carrying a completion tag, data, a 2-bit status, and an end-of-packet marker, used to return read data (CplD) to the AXI R channel.

3.2 Functional Scope Implemented

- One AXI4 transaction outstanding at a time (write and read are mutually exclusive while a transaction is in flight); write traffic is prioritized over read when both are pending.
- INCR and FIXED AXI bursts are supported; each AXI beat is converted into exactly one PCIe MemWr or MemRd TLP request, with the address incrementing between beats for INCR bursts.
- Only full-byte-write strobes (`WSTRB` = all-ones) are supported; a partial or malformed WSTRB, a WLAST that does not align with `AWLEN`, or an unsupported burst type is rejected with `BRESP = SLVERR` and no PCIe TLP is issued.
- An invalid AXI read attribute (unsupported size/burst) is rejected with `RRESP = SLVERR` without issuing a PCIe request.

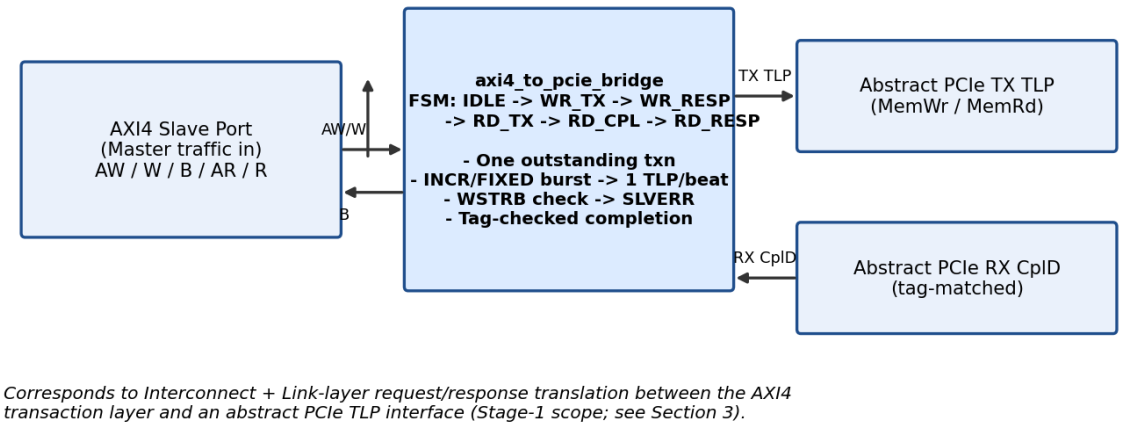


Figure 6: Stage-1 block diagram: `axi4_to_pcie_bridge` sits between the AXI4 slave port and an abstract PCIe TLP request/completion interface.

- PCIe completion status is mapped onto `RRESP` (OKAY/SLVERR), and a completion is only consumed when its tag matches the currently outstanding read – an unrelated or stale completion is left un-acknowledged rather than corrupting the AXI R channel.

3.3 Bridge FSM

The bridge is implemented as a single Mealy-style FSM (Figure 7) with six states: `IDLE`, `WR_TX`, `WR_RESP`, `RD_TX`, `RD_CPL`, `RD_RESP`. Ready signals (`AWREADY`, `WREADY`, `ARREADY`, `rx_cpl_ready`) are driven combinationally from the current state so that they are valid before the corresponding clock edge, as required by the AXI4 handshake protocol.

4 Verification Strategy

A self-checking, directed testbench (`tb_axi4_to_pcie_bridge`, in `tb_bridge.v`) drives the DUT and checks results with `pass/fail` tasks that tally a running pass/fail count, avoiding reliance on manual waveform inspection for pass/fail determination. Ten scenarios were exercised:

1. Reset – all valid outputs deasserted after `rst_n` is released.
2. Single-beat AXI write → single MemWr TLP → BRESP=OKAY.
3. AW and W arriving on independent cycles (out-of-order channel timing).
4. 3-beat AXI INCR write burst → 3 MemWr TLPs with correctly incrementing address.
5. Partial `WSTRB` → SLVERR, and no PCIe TLP is generated.
6. Single-beat AXI read with a successful completion, including TX backpressure (`tx_tlp_ready` deasserted) to confirm `tx_tlp_valid` is held stably.
7. 2-beat AXI INCR read burst with two separate completions, checking RLAST placement and per-beat address increment.
8. PCIe completion error status (≠ OKAY) → AXI `RRESP = SLVERR`.

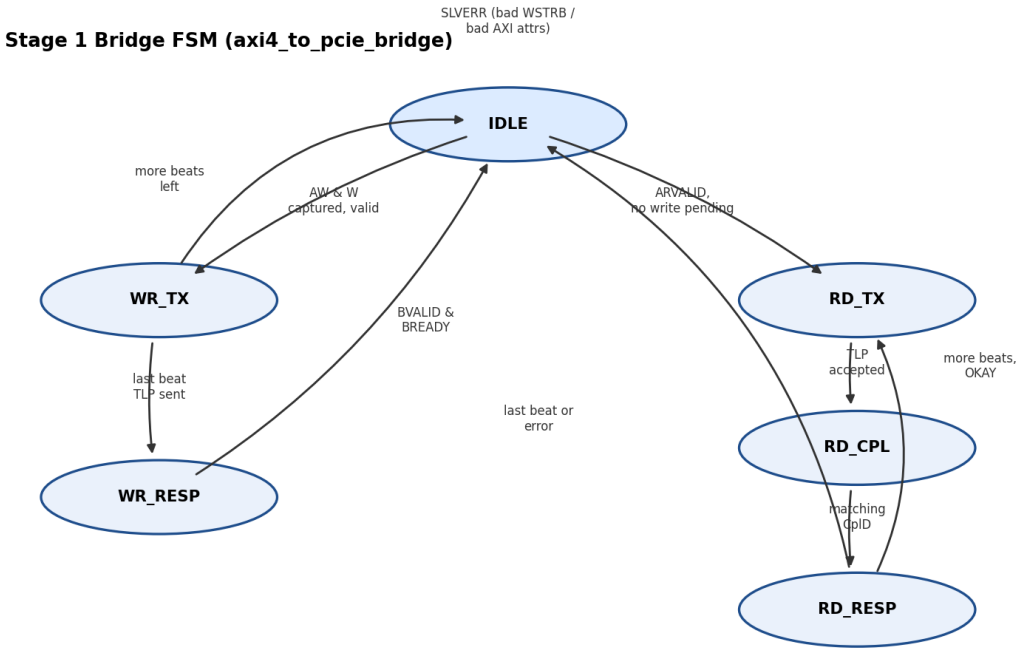


Figure 7: Bridge finite-state machine. Self-loop on `IDLE` represents SLVERR generation for malformed write/read attributes without leaving the idle state's request-capture logic.

9. A completion with the wrong tag is correctly rejected (not consumed, not forwarded to the AXI R channel); the correct completion is still accepted afterward.

Result Summary

Metric	Value
Test scenarios	10
Total self-checks	69
Checks passed	69
Checks failed	0
Simulator	Icarus Verilog 12.0 (`iverilog/vvp`)

The complete console log ends with:

```
============================================================
Stage-1 Verification Summary
PASS checks : 69
FAIL checks : 0
RESULT     : ALL TESTS PASSED
============================================================
```

5 Simulation Waveforms

Waveforms below were captured from the VCD dump (`bridge_stage1.vcd`) produced by the testbench and are reproduced directly from simulation, not hand-drawn.

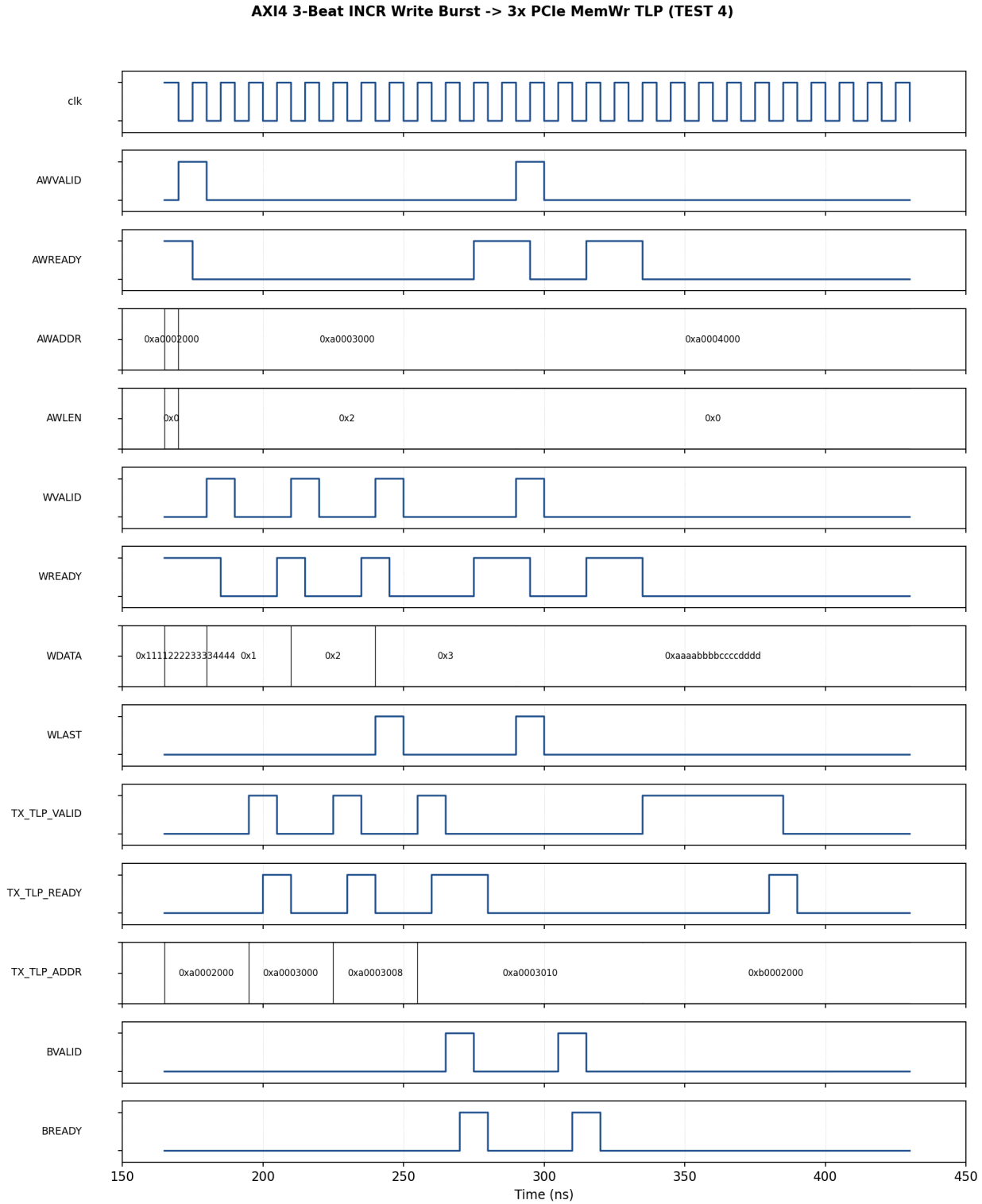


Figure 8: Test 4: a 3-beat AXI4 INCR write burst (`AWADDR=0xA0003000`, `AWLEN=2`) is broken into three independent MemWr TLPs on the abstract PCIe TX interface, one per AXI beat, with `TX_TLP_ADDR` incrementing by 8 bytes per beat and `BVALID` asserted only after the final beat.

6 Reset, Error and Corner-Case Behavior

- **Reset:** on `rst_n` deassertion, the FSM returns to `IDLE` and all valid outputs (`BVALID`, `RVALID`, `tx_tlp_valid`) are cleared; verified directly by Test 1.

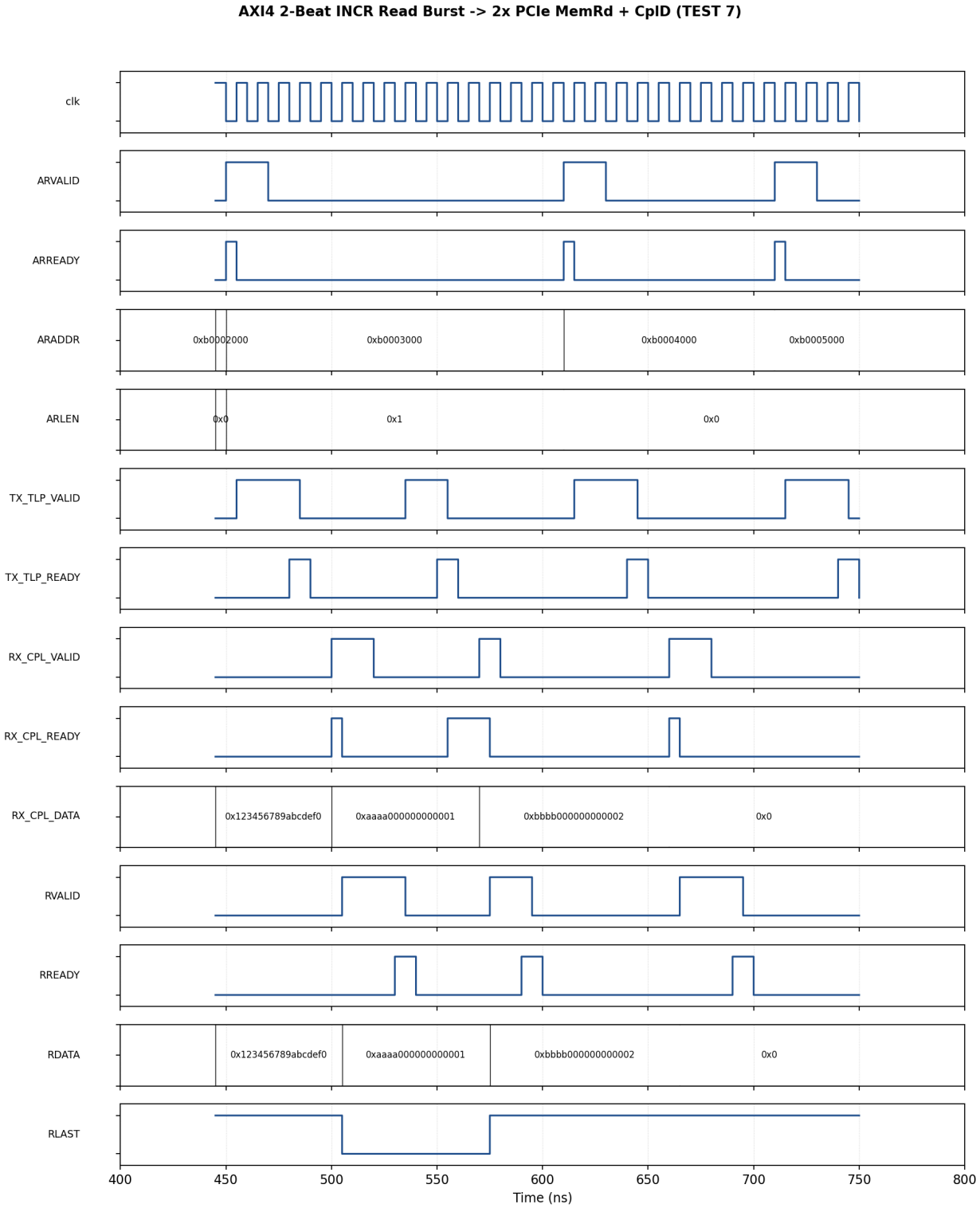


Figure 9: Test 7: a 2-beat AXI4 INCR read burst (`ARADDR=0xB0003000`, `ARLEN=1`) issues one MemRd TLP per beat and waits for the matching completion before advancing; `RLAST` is deasserted on the first beat and asserted on the second, and each beat's `RDATA` matches the corresponding `RX_CPL_DATA` payload.

- **Malformed write:** partial byte strobes are rejected with SLVERR before any PCIe traffic is generated (Test 5), preventing a partially-written AXI beat from silently reaching the PCIe side.

- **Malformed read:** an unsupported AXI read `ARSIZE/ARBURST` is rejected with SLVERR without issuing a PCIe MemRd request.
- **Completion/tag integrity:** the bridge only asserts `rx_cpl_ready` when the incoming completion's tag matches the currently outstanding read ID, so a stale or unrelated completion cannot be misrouted onto the AXI R channel (Test 9).
- **Backpressure:** `tx_tlp_valid` is held stably (not glitched or dropped) while `tx_tlp_ready` is low, satisfying the AXI-style valid/ready stability rule reused for the abstract PCIe interface (Test 6).

7 Implementation Scope vs. Full Architecture

Architecture Layer	Representative Function	Stage-1 Status
AXI4 / AXI4-Stream Interface	Transaction/data ingress and egress	**Implemented** (AXI4 MM slave side)
Interconnect Layer	Routing, crossbar operation and arbitration	**Partially implemented** (single-master request scheduling inside the bridge FSM only; no multi-master crossbar)
Link Layer	Packetization, framing, CRC/error-handling	**Abstracted** (TLP-level valid/ready interface modeled; 256-byte FIU packetizer and CRC/FEC not implemented)
PCIe Protocol-to-PHY Layer	Protocol adaptation, lane striping, PHY-facing interface	**Out of scope** for Stage 1

8 Limitations, Assumptions and Future Improvements

Assumptions. The AXI4 and PCIe TLP interfaces are assumed to be driven by well-formed, protocol-legal masters except where a scenario intentionally exercises an error path; a single AXI4 data width (default 64-bit) is exercised in this report, though the module is parameterized for other widths; the abstract TLP/CplD interface stands in for the eventual Link-Layer packetizer output, so end-to-end 256-byte-flow-unit framing and FEC/CRC are not exercised end-to-end in this stage.

Limitations. Only one AXI4 transaction is outstanding at a time – there is no transaction-ID-based reordering or pipelining of multiple in-flight requests; only full-byte write strobes are supported; the AXI4-Stream ingress/egress, the multi-master crossbar/arbiter, the FEC/CRC packetizer, and the PAM4/multi-lane PHY layer are not yet implemented and are represented only architecturally in Section 3.

Future improvements (Stage 2 direction). Add the AXI4-Stream front end and connect it to this bridge's TLP interface through an explicit Link-Layer packetizer with 256-byte Flow Information Units and CRC generation/checking; extend the FSM to support multiple outstanding transactions using the existing tag field; add a central crossbar/arbiter ahead of the bridge to support multiple AXI masters as called for by the Interconnect Layer; and begin synthesis and OpenROAD/OpenLane physical-implementation experiments on the blocks completed here.

A Appendix A: RTL Source (axi4_to_pcie_bridge, comments removed)

```
`timescale 1ns/1ps

module axi4_to_pcie_bridge #(
    parameter integer DATA_WIDTH = 64,
    parameter integer ADDR_WIDTH = 32,
    parameter integer ID_WIDTH = 4
)(
    input  wire                     clk,
    input  wire                     rst_n,

    input  wire [ID_WIDTH-1:0]      s_axi_awid,
    input  wire [ADDR_WIDTH-1:0]    s_axi_awaddr,
    input  wire [7:0]               s_axi_awlen,
    input  wire [2:0]               s_axi_awsize,
    input  wire [1:0]               s_axi_awburst,
    input  wire                     s_axi_awvalid,
    output reg                      s_axi_awready,

    input  wire [DATA_WIDTH-1:0]    s_axi_wdata,
    input  wire [(DATA_WIDTH/8)-1:0] s_axi_wstrb,
    input  wire                     s_axi_wlast,
    input  wire                     s_axi_wvalid,
    output reg                      s_axi_wready,

    output reg [ID_WIDTH-1:0]       s_axi_bid,
    output reg [1:0]                s_axi_bresp,
    output reg                      s_axi_bvalid,
    input  wire                     s_axi_bready,

    input  wire [ID_WIDTH-1:0]      s_axi_arid,
    input  wire [ADDR_WIDTH-1:0]    s_axi_araddr,
    input  wire [7:0]               s_axi_arlen,
    input  wire [2:0]               s_axi_arsize,
    input  wire [1:0]               s_axi_arburst,
    input  wire                     s_axi_arvalid,
    output reg                      s_axi_arready,

    output reg [ID_WIDTH-1:0]       s_axi_rid,
    output reg [DATA_WIDTH-1:0]     s_axi_rdata,
    output reg [1:0]                s_axi_rresp,
    output reg                      s_axi_rlast,
    output reg                      s_axi_rvalid,
    input  wire                     s_axi_rready,

    output reg                      tx_tlp_valid,
    input  wire                     tx_tlp_ready,
    output reg [4:0]                tx_tlp_type,
    output reg [ADDR_WIDTH-1:0]     tx_tlp_addr,
    output reg [9:0]                tx_tlp_length,
    output reg [7:0]                tx_tlp_tag,
    output reg [DATA_WIDTH-1:0]     tx_tlp_data,
    output reg                      tx_tlp_sop,
    output reg                      tx_tlp_eop,

    input  wire                     rx_cpl_valid,
    output reg                      rx_cpl_ready,
    input  wire [7:0]               rx_cpl_tag,
    input  wire [DATA_WIDTH-1:0]    rx_cpl_data,
    input  wire [1:0]               rx_cpl_status,
    input  wire                     rx_cpl_eop
);

```

```
    localparam [4:0] TLP_MEM_WR = 5'b00000;
    localparam [4:0] TLP_MEM_RD = 5'b00001;

    localparam [1:0] AXI_OKAY = 2'b00;
    localparam [1:0] AXI_SLVERR = 2'b10;

    localparam [1:0] AXI_BURST_FIXED = 2'b00;
    localparam [1:0] AXI_BURST_INCR = 2'b01;

    localparam integer DATA_BYTES = DATA_WIDTH / 8;
    localparam integer DATA_DW = DATA_WIDTH / 32;
    localparam [2:0] AXI_SIZE  = $clog2(DATA_BYTES);

    localparam [3:0]
        ST_IDLE      = 4'd0,
        ST_WR_TX     = 4'd1,
        ST_WR_RESP   = 4'd2,
        ST_RD_TX     = 4'd3,
        ST_RD_CPL    = 4'd4,
        ST_RD_RESP   = 4'd5;

    reg [3:0] state;

    reg                       aw_hold;
    reg                       w_hold;
    reg [ID_WIDTH-1:0]        wr_id;
    reg [ADDR_WIDTH-1:0]      wr_addr;
    reg [7:0]                 wr_beats_left;
    reg [1:0]                 wr_burst;
    reg [2:0]                 wr_size;
    reg [DATA_WIDTH-1:0]      wr_data;
    reg [(DATA_WIDTH/8)-1:0]  wr_strb;
    reg                       wr_last;

    reg [ID_WIDTH-1:0]        rd_id;
    reg [ADDR_WIDTH-1:0]      rd_addr;
    reg [7:0]                 rd_beats_left;
    reg [1:0]                 rd_burst;
    reg [2:0]                 rd_size;

    always @* begin
        s_axi_awready = 1'b0;
        s_axi_wready = 1'b0;
        s_axi_arready = 1'b0;
        rx_cpl_ready = 1'b0;

        if (state == ST_IDLE) begin

            s_axi_awready = !aw_hold;
            s_axi_wready = !w_hold;

            if (!aw_hold && !w_hold && !s_axi_awvalid && !s_axi_wvalid &&
                s_axi_arvalid) begin
                s_axi_arready = 1'b1;
            end
        end

        if ((state == ST_RD_CPL) &&
            (rx_cpl_tag == {{(8-ID_WIDTH){1'b0}}, rd_id})) begin
            rx_cpl_ready = 1'b1;
        end
    end

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
```

```
            state         <= ST_IDLE;
            aw_hold       <= 1'b0;
            w_hold        <= 1'b0;

            wr_id         <= {ID_WIDTH{1'b0}};
            wr_addr       <= {ADDR_WIDTH{1'b0}};
            wr_beats_left <= 8'd0;
            wr_burst      <= AXI_BURST_INCR;
            wr_size       <= AXI_SIZE;
            wr_data       <= {DATA_WIDTH{1'b0}};
            wr_strb       <= {(DATA_WIDTH/8){1'b0}};
            wr_last       <= 1'b0;

            rd_id         <= {ID_WIDTH{1'b0}};
            rd_addr       <= {ADDR_WIDTH{1'b0}};
            rd_beats_left <= 8'd0;
            rd_burst      <= AXI_BURST_INCR;
            rd_size       <= AXI_SIZE;

            s_axi_bvalid  <= 1'b0;
            s_axi_bid     <= {ID_WIDTH{1'b0}};
            s_axi_bresp   <= AXI_OKAY;

            s_axi_rvalid  <= 1'b0;
            s_axi_rid     <= {ID_WIDTH{1'b0}};
            s_axi_rdata   <= {DATA_WIDTH{1'b0}};
            s_axi_rresp   <= AXI_OKAY;
            s_axi_rlast   <= 1'b0;

            tx_tlp_valid  <= 1'b0;
            tx_tlp_type   <= 5'd0;
            tx_tlp_addr   <= {ADDR_WIDTH{1'b0}};
            tx_tlp_length <= 10'd0;
            tx_tlp_tag    <= 8'd0;
            tx_tlp_data   <= {DATA_WIDTH{1'b0}};
            tx_tlp_sop    <= 1'b0;
            tx_tlp_eop    <= 1'b0;
        end else begin
            case (state)

                ST_IDLE: begin
                    tx_tlp_valid <= 1'b0;
                    s_axi_bvalid <= 1'b0;

                    if (s_axi_awvalid && s_axi_awready) begin
                        aw_hold      <= 1'b1;
                        wr_id        <= s_axi_awid;
                        wr_addr      <= s_axi_awaddr;
                        wr_beats_left <= s_axi_awlen + 8'd1;
                        wr_burst     <= s_axi_awburst;
                        wr_size      <= s_axi_awsize;
                    end

                    if (s_axi_wvalid && s_axi_wready) begin
                        w_hold <= 1'b1;
                        wr_data <= s_axi_wdata;
                        wr_strb <= s_axi_wstrb;
                        wr_last <= s_axi_wlast;
                    end

                    if (aw_hold && w_hold) begin
                        if ((wr_size != AXI_SIZE) ||
                            ((wr_burst != AXI_BURST_FIXED) &&
                             (wr_burst != AXI_BURST_INCR)) ||
                            (wr_strb != {(DATA_WIDTH/8){1'b1}}) ||
```

```
                        (wr_last != (wr_beats_left == 8'd1))) begin

                        s_axi_bid   <= wr_id;
                        s_axi_bresp <= AXI_SLVERR;
                        s_axi_bvalid <= 1'b1;
                        aw_hold     <= 1'b0;
                        w_hold      <= 1'b0;
                        state       <= ST_WR_RESP;
                    end else begin
                        tx_tlp_valid <= 1'b1;
                        tx_tlp_type  <= TLP_MEM_WR;
                        tx_tlp_addr  <= wr_addr;
                        tx_tlp_length <= DATA_DW;
                        tx_tlp_tag   <= {{(8-ID_WIDTH){1'b0}}, wr_id};
                        tx_tlp_data  <= wr_data;
                        tx_tlp_sop   <= 1'b1;
                        tx_tlp_eop   <= 1'b1;
                        state        <= ST_WR_TX;
                    end
                end

                else if (s_axi_arvalid && s_axi_arready) begin
                    rd_id        <= s_axi_arid;
                    rd_addr      <= s_axi_araddr;
                    rd_beats_left <= s_axi_arlen + 8'd1;
                    rd_burst     <= s_axi_arburst;
                    rd_size      <= s_axi_arsize;

                    if ((s_axi_arsize != AXI_SIZE) ||
                        ((s_axi_arburst != AXI_BURST_FIXED) &&
                         (s_axi_arburst != AXI_BURST_INCR))) begin

                        s_axi_rid   <= s_axi_arid;
                        s_axi_rdata <= {DATA_WIDTH{1'b0}};
                        s_axi_rresp <= AXI_SLVERR;
                        s_axi_rlast <= 1'b1;
                        s_axi_rvalid <= 1'b1;
                        state       <= ST_RD_RESP;
                    end else begin
                        tx_tlp_valid <= 1'b1;
                        tx_tlp_type  <= TLP_MEM_RD;
                        tx_tlp_addr  <= s_axi_araddr;
                        tx_tlp_length <= DATA_DW;
                        tx_tlp_tag   <= {{(8-ID_WIDTH){1'b0}}, s_axi_arid};
                        tx_tlp_data  <= {DATA_WIDTH{1'b0}};
                        tx_tlp_sop   <= 1'b1;
                        tx_tlp_eop   <= 1'b1;
                        state        <= ST_RD_TX;
                    end
                end
            end

            ST_WR_TX: begin
                if (tx_tlp_valid && tx_tlp_ready) begin
                    tx_tlp_valid <= 1'b0;
                    w_hold      <= 1'b0;

                    if (wr_beats_left == 8'd1) begin
                        aw_hold     <= 1'b0;
                        s_axi_bid   <= wr_id;
                        s_axi_bresp <= AXI_OKAY;
                        s_axi_bvalid <= 1'b1;
                        state       <= ST_WR_RESP;
                    end else begin
                        wr_beats_left <= wr_beats_left - 8'd1;
```

```
                if (wr_burst == AXI_BURST_INCR)
                    wr_addr <= wr_addr + DATA_BYTES;
                state <= ST_IDLE;
            end
        end
    end

    ST_WR_RESP: begin
        if (s_axi_bvalid && s_axi_bready) begin
            s_axi_bvalid <= 1'b0;
            state       <= ST_IDLE;
        end
    end

    ST_RD_TX: begin
        if (tx_tlp_valid && tx_tlp_ready) begin
            tx_tlp_valid <= 1'b0;
            state       <= ST_RD_CPL;
        end
    end

    ST_RD_CPL: begin
        if (rx_cpl_valid && rx_cpl_ready) begin
            s_axi_rid   <= rd_id;
            s_axi_rdata <= rx_cpl_data;
            s_axi_rresp <= (rx_cpl_status == 2'b00) ?
                           AXI_OKAY : AXI_SLVERR;
            s_axi_rlast <= (rd_beats_left == 8'd1);

            if (rx_cpl_eop != (rd_beats_left == 8'd1))
                s_axi_rresp <= AXI_SLVERR;

            s_axi_rvalid <= 1'b1;
            state       <= ST_RD_RESP;
        end
    end

    ST_RD_RESP: begin
        if (s_axi_rvalid && s_axi_rready) begin
            s_axi_rvalid <= 1'b0;

            if ((rd_beats_left > 8'd1) &&
                (s_axi_rresp == AXI_OKAY)) begin
                rd_beats_left <= rd_beats_left - 8'd1;

                if (rd_burst == AXI_BURST_INCR)
                    rd_addr <= rd_addr + DATA_BYTES;

                tx_tlp_valid <= 1'b1;
                tx_tlp_type  <= TLP_MEM_RD;
                tx_tlp_addr  <= (rd_burst == AXI_BURST_INCR) ?
                                (rd_addr + DATA_BYTES) : rd_addr;
                tx_tlp_length <= DATA_DW;
                tx_tlp_tag   <= {{(8-ID_WIDTH){1'b0}}, rd_id};
                tx_tlp_data  <= {DATA_WIDTH{1'b0}};
                tx_tlp_sop   <= 1'b1;
                tx_tlp_eop   <= 1'b1;
                state        <= ST_RD_TX;
            end else begin
                state <= ST_IDLE;
            end
        end
    end

    default: begin
```

```
                    state <= ST_IDLE;
                end
            endcase
        end
    end

endmodule
```

B Appendix B: Testbench Source (comments removed)

```
`timescale 1ns/1ps

module tb_axi4_to_pcie_bridge;

    parameter DATA_WIDTH = 64;
    parameter ADDR_WIDTH = 32;
    parameter ID_WIDTH = 4;

    reg clk;
    reg rst_n;

    reg [ID_WIDTH-1:0] s_axi_awid;
    reg [ADDR_WIDTH-1:0] s_axi_awaddr;
    reg [7:0] s_axi_awlen;
    reg [2:0] s_axi_awsize;
    reg [1:0] s_axi_awburst;
    reg s_axi_awvalid;
    wire s_axi_awready;

    reg [DATA_WIDTH-1:0] s_axi_wdata;
    reg [(DATA_WIDTH/8)-1:0] s_axi_wstrb;
    reg s_axi_wlast;
    reg s_axi_wvalid;
    wire s_axi_wready;

    wire [ID_WIDTH-1:0] s_axi_bid;
    wire [1:0] s_axi_bresp;
    wire s_axi_bvalid;
    reg s_axi_bready;

    reg [ID_WIDTH-1:0] s_axi_arid;
    reg [ADDR_WIDTH-1:0] s_axi_araddr;
    reg [7:0] s_axi_arlen;
    reg [2:0] s_axi_arsize;
    reg [1:0] s_axi_arburst;
    reg s_axi_arvalid;
    wire s_axi_arready;

    wire [ID_WIDTH-1:0] s_axi_rid;
    wire [DATA_WIDTH-1:0] s_axi_rdata;
    wire [1:0] s_axi_rresp;
    wire s_axi_rlast;
    wire s_axi_rvalid;
    reg s_axi_rready;

    wire tx_tlp_valid;
    reg tx_tlp_ready;
    wire [4:0] tx_tlp_type;
    wire [ADDR_WIDTH-1:0] tx_tlp_addr;
    wire [9:0] tx_tlp_length;
    wire [7:0] tx_tlp_tag;
    wire [DATA_WIDTH-1:0] tx_tlp_data;
    wire tx_tlp_sop;
    wire tx_tlp_eop;

    reg rx_cpl_valid;
    wire rx_cpl_ready;
    reg [7:0] rx_cpl_tag;
    reg [DATA_WIDTH-1:0] rx_cpl_data;
    reg [1:0] rx_cpl_status;
    reg rx_cpl_eop;

```

```
    integer errors;
    integer pass_count;

    localparam [4:0] TLP_MEM_WR = 5'b00000;
    localparam [4:0] TLP_MEM_RD = 5'b00001;

    axi4_to_pcie_bridge #(
        .DATA_WIDTH(DATA_WIDTH),
        .ADDR_WIDTH(ADDR_WIDTH),
        .ID_WIDTH(ID_WIDTH)
    ) dut (
        .clk(clk), .rst_n(rst_n),
        .s_axi_awid(s_axi_awid), .s_axi_awaddr(s_axi_awaddr),
        .s_axi_awlen(s_axi_awlen), .s_axi_awsize(s_axi_awsize),
        .s_axi_awburst(s_axi_awburst), .s_axi_awvalid(s_axi_awvalid),
        .s_axi_awready(s_axi_awready),
        .s_axi_wdata(s_axi_wdata), .s_axi_wstrb(s_axi_wstrb),
        .s_axi_wlast(s_axi_wlast), .s_axi_wvalid(s_axi_wvalid),
        .s_axi_wready(s_axi_wready),
        .s_axi_bid(s_axi_bid), .s_axi_bresp(s_axi_bresp),
        .s_axi_bvalid(s_axi_bvalid), .s_axi_bready(s_axi_bready),
        .s_axi_arid(s_axi_arid), .s_axi_araddr(s_axi_araddr),
        .s_axi_arlen(s_axi_arlen), .s_axi_arsize(s_axi_arsize),
        .s_axi_arburst(s_axi_arburst), .s_axi_arvalid(s_axi_arvalid),
        .s_axi_arready(s_axi_arready),
        .s_axi_rid(s_axi_rid), .s_axi_rdata(s_axi_rdata),
        .s_axi_rresp(s_axi_rresp), .s_axi_rlast(s_axi_rlast),
        .s_axi_rvalid(s_axi_rvalid), .s_axi_rready(s_axi_rready),
        .tx_tlp_valid(tx_tlp_valid), .tx_tlp_ready(tx_tlp_ready),
        .tx_tlp_type(tx_tlp_type), .tx_tlp_addr(tx_tlp_addr),
        .tx_tlp_length(tx_tlp_length), .tx_tlp_tag(tx_tlp_tag),
        .tx_tlp_data(tx_tlp_data), .tx_tlp_sop(tx_tlp_sop),
        .tx_tlp_eop(tx_tlp_eop),
        .rx_cpl_valid(rx_cpl_valid), .rx_cpl_ready(rx_cpl_ready),
        .rx_cpl_tag(rx_cpl_tag), .rx_cpl_data(rx_cpl_data),
        .rx_cpl_status(rx_cpl_status), .rx_cpl_eop(rx_cpl_eop)
    );

    always #5 clk = ~clk;

    task fail;
        input [8*100-1:0] msg;
        begin
            errors = errors + 1;
            $display("FAIL: %0s", msg);
        end
    endtask

    task pass;
        input [8*100-1:0] msg;
        begin
            pass_count = pass_count + 1;
            $display("PASS: %0s", msg);
        end
    endtask

    task drive_defaults;
        begin
            s_axi_awid = 0; s_axi_awaddr = 0; s_axi_awlen = 0;
            s_axi_awsize = 3'b011; s_axi_awburst = 2'b01; s_axi_awvalid = 0;
            s_axi_wdata = 0; s_axi_wstrb = 8'hFF; s_axi_wlast = 0; s_axi_wvalid = 0;
            s_axi_bready = 0;
            s_axi_arid = 0; s_axi_araddr = 0; s_axi_arlen = 0;
            s_axi_arsize = 3'b011; s_axi_arburst = 2'b01; s_axi_arvalid = 0;
            s_axi_rready = 0;
```

```
            tx_tlp_ready = 0;
            rx_cpl_valid = 0; rx_cpl_tag = 0; rx_cpl_data = 0;
            rx_cpl_status = 0; rx_cpl_eop = 0;
        end
    endtask

    task reset_dut;
        begin
            rst_n = 0;
            repeat (3) @(posedge clk);
            if (s_axi_bvalid || s_axi_rvalid || tx_tlp_valid)
                fail("Reset did not clear valid outputs");
            rst_n = 1;
            @(posedge clk);
        end
    endtask

    task axi_write_single;
        input [ID_WIDTH-1:0] id;
        input [ADDR_WIDTH-1:0] addr;
        input [DATA_WIDTH-1:0] data;
        input [DATA_WIDTH/8-1:0] strb;
        begin
            @(negedge clk);
            s_axi_awid = id; s_axi_awaddr = addr; s_axi_awlen = 0;
            s_axi_awsize = 3'b011; s_axi_awburst = 2'b01; s_axi_awvalid = 1;
            s_axi_wdata = data; s_axi_wstrb = strb; s_axi_wlast = 1; s_axi_wvalid = 1;
            while (!(s_axi_awready && s_axi_wready)) @(posedge clk);
            @(posedge clk);
            @(negedge clk);
            s_axi_awvalid = 0;
            s_axi_wvalid = 0;
            s_axi_wlast = 0;
        end
    endtask

    task check_write_tlp;
        input [ID_WIDTH-1:0] id;
        input [ADDR_WIDTH-1:0] addr;
        input [DATA_WIDTH-1:0] data;
        begin
            wait (tx_tlp_valid);
            if (tx_tlp_type !== TLP_MEM_WR) fail("Write TLP type incorrect");
            else pass("Write TLP type = MemWr");
            if (tx_tlp_addr !== addr) fail("Write TLP address incorrect");
            else pass("Write TLP address correct");
            if (tx_tlp_length !== 10'd2) fail("Write TLP length incorrect");
            else pass("Write TLP length = 2 DW");
            if (tx_tlp_tag !== {4'b0,id}) fail("Write TLP tag incorrect");
            else pass("Write TLP tag correct");
            if (tx_tlp_data !== data) fail("Write TLP payload incorrect");
            else pass("Write TLP payload correct");
            if (!(tx_tlp_sop && tx_tlp_eop)) fail("Write TLP SOP/EOP incorrect");
            else pass("Write TLP SOP/EOP correct");
        end
    endtask

    task complete_write;
        input [ID_WIDTH-1:0] id;
        input [1:0] expected_resp;
        begin
            @(negedge clk);
            tx_tlp_ready = 1;
            wait (s_axi_bvalid);
            if (s_axi_bid !== id) fail("BID incorrect");
```

```
            if (s_axi_bresp !== expected_resp) fail("BRESP incorrect");
            else pass("AXI write response correct");
            @(negedge clk);
            s_axi_bready = 1;
            @(posedge clk);
            @(negedge clk);
            s_axi_bready = 0;
            tx_tlp_ready = 0;
        end
    endtask

    task axi_read_single;
        input [ID_WIDTH-1:0] id;
        input [ADDR_WIDTH-1:0] addr;
        begin
            @(negedge clk);
            s_axi_arid = id; s_axi_araddr = addr; s_axi_arlen = 0;
            s_axi_arsize = 3'b011; s_axi_arburst = 2'b01; s_axi_arvalid = 1;
            while (!s_axi_arready) @(posedge clk);
            @(posedge clk);
            @(negedge clk);
            s_axi_arvalid = 0;
        end
    endtask

    task check_read_tlp;
        input [ID_WIDTH-1:0] id;
        input [ADDR_WIDTH-1:0] addr;
        begin
            wait (tx_tlp_valid);
            if (tx_tlp_type !== TLP_MEM_RD) fail("Read TLP type incorrect");
            else pass("Read TLP type = MemRd");
            if (tx_tlp_addr !== addr) fail("Read TLP address incorrect");
            else pass("Read TLP address correct");
            if (tx_tlp_length !== 10'd2) fail("Read TLP length incorrect");
            else pass("Read TLP length = 2 DW");
            if (tx_tlp_tag !== {4'b0,id}) fail("Read TLP tag incorrect");
            else pass("Read TLP tag correct");
            if (!(tx_tlp_sop && tx_tlp_eop)) fail("Read TLP SOP/EOP incorrect");
            else pass("Read TLP SOP/EOP correct");
        end
    endtask

    task send_completion;
        input [7:0] tag;
        input [DATA_WIDTH-1:0] data;
        input [1:0] status;
        input eop;
        begin
            @(negedge clk);
            rx_cpl_tag = tag;
            rx_cpl_data = data;
            rx_cpl_status = status;
            rx_cpl_eop = eop;
            rx_cpl_valid = 1;
            while (!rx_cpl_ready) @(posedge clk);
            @(posedge clk);
            @(negedge clk);
            rx_cpl_valid = 0;
        end
    endtask

    initial begin
        $dumpfile("bridge_stage1.vcd");
        $dumpvars(0, tb_axi4_to_pcie_bridge);
```

```
        clk = 0;
        errors = 0;
        pass_count = 0;
        drive_defaults;

        $display("\n=== TEST 1: Reset ===");
        reset_dut;
        pass("Reset sequence completed");

        $display("\n=== TEST 2: Single-beat AXI Write ===");
        axi_write_single(4'h1, 32'hA000_1000, 64'hDEAD_BEEF_CAFE_0001, 8'hFF);
        check_write_tlp(4'h1, 32'hA000_1000, 64'hDEAD_BEEF_CAFE_0001);
        complete_write(4'h1, 2'b00);

        $display("\n=== TEST 3: Independent AW/W Timing ===");
        @(negedge clk);
        s_axi_awid = 4'h2; s_axi_awaddr = 32'hA000_2000; s_axi_awlen = 0;
        s_axi_awsize = 3'b011; s_axi_awburst = 2'b01; s_axi_awvalid = 1;
        while (!s_axi_awready) @(posedge clk);
        @(posedge clk);
        @(negedge clk);
        s_axi_awvalid = 0;
        repeat (2) @(posedge clk);
        @(negedge clk);
        s_axi_wdata = 64'h1111_2222_3333_4444; s_axi_wstrb = 8'hFF;
        s_axi_wlast = 1; s_axi_wvalid = 1;
        while (!s_axi_wready) @(posedge clk);
        @(posedge clk);
        @(negedge clk);
        s_axi_wvalid = 0; s_axi_wlast = 0;
        check_write_tlp(4'h2, 32'hA000_2000, 64'h1111_2222_3333_4444);
        complete_write(4'h2, 2'b00);
        pass("AW and W accepted independently");

        $display("\n=== TEST 4: 3-beat AXI Write Burst ===");
        @(negedge clk);
        s_axi_awid = 4'h3; s_axi_awaddr = 32'hA000_3000; s_axi_awlen = 2;
        s_axi_awsize = 3'b011; s_axi_awburst = 2'b01; s_axi_awvalid = 1;
        while (!s_axi_awready) @(posedge clk);
        @(posedge clk);
        @(negedge clk); s_axi_awvalid = 0;

        s_axi_wdata = 64'h0000_0000_0000_0001; s_axi_wstrb = 8'hFF;
        s_axi_wlast = 0; s_axi_wvalid = 1;
        while (!s_axi_wready) @(posedge clk);
        @(posedge clk); @(negedge clk); s_axi_wvalid = 0;
        check_write_tlp(4'h3, 32'hA000_3000, 64'h1);
        @(negedge clk); tx_tlp_ready = 1;
        wait (tx_tlp_valid); @(posedge clk); @(negedge clk); tx_tlp_ready = 0;

        s_axi_wdata = 64'h0000_0000_0000_0002; s_axi_wstrb = 8'hFF;
        s_axi_wlast = 0; s_axi_wvalid = 1;
        while (!s_axi_wready) @(posedge clk);
        @(posedge clk); @(negedge clk); s_axi_wvalid = 0;
        check_write_tlp(4'h3, 32'hA000_3008, 64'h2);
        @(negedge clk); tx_tlp_ready = 1;
        wait (tx_tlp_valid); @(posedge clk); @(negedge clk); tx_tlp_ready = 0;

        s_axi_wdata = 64'h0000_0000_0000_0003; s_axi_wstrb = 8'hFF;
        s_axi_wlast = 1; s_axi_wvalid = 1;
        while (!s_axi_wready) @(posedge clk);
        @(posedge clk); @(negedge clk); s_axi_wvalid = 0; s_axi_wlast = 0;
        check_write_tlp(4'h3, 32'hA000_3010, 64'h3);
        complete_write(4'h3, 2'b00);
```

```
        pass("3-beat AXI INCR burst converted to 3 MemWr TLPs");

        $display("\n=== TEST 5: Partial WSTRB Error ===");
        axi_write_single(4'h4, 32'hA000_4000, 64'hAAAA_BBBB_CCCC_DDDD, 8'h0F);
        wait (s_axi_bvalid);
        if (tx_tlp_valid) fail("Partial WSTRB incorrectly generated a PCIe TLP");
        if (s_axi_bresp !== 2'b10) fail("Partial WSTRB did not return SLVERR");
        else pass("Partial WSTRB correctly rejected with SLVERR");
        @(negedge clk); s_axi_bready = 1; @(posedge clk); @(negedge clk); s_axi_bready = 0;

        $display("\n=== TEST 6: Single-beat AXI Read + Successful CplD ===");
        axi_read_single(4'h5, 32'hB000_2000);
        check_read_tlp(4'h5, 32'hB000_2000);

        if (!tx_tlp_valid) fail("Read TLP valid not asserted");
        @(negedge clk); tx_tlp_ready = 0;
        repeat (2) @(posedge clk);
        if (!tx_tlp_valid) fail("TX valid was not held under backpressure");
        else pass("TX valid held under backpressure");
        @(negedge clk); tx_tlp_ready = 1;
        @(posedge clk); @(negedge clk); tx_tlp_ready = 0;

        send_completion(8'h05, 64'h1234_5678_9ABC_DEF0, 2'b00, 1'b1);
        wait (s_axi_rvalid);
        if (s_axi_rid !== 4'h5) fail("RID incorrect for successful CplD");
        if (s_axi_rdata !== 64'h1234_5678_9ABC_DEF0) fail("RDATA incorrect for successful CplD");
        if (s_axi_rresp !== 2'b00) fail("RRESP should be OKAY");
        if (!s_axi_rlast) fail("RLAST should be asserted for single-beat read");
        else pass("Successful CplD correctly returned on AXI R channel");
        @(negedge clk); s_axi_rready = 1; @(posedge clk); @(negedge clk); s_axi_rready = 0;

        $display("\n=== TEST 7: Read Burst with Multiple Completions ===");
        @(negedge clk);
        s_axi_arid = 4'h6; s_axi_araddr = 32'hB000_3000; s_axi_arlen = 1;
        s_axi_arsize = 3'b011; s_axi_arburst = 2'b01; s_axi_arvalid = 1;
        while (!s_axi_arready) @(posedge clk);
        @(posedge clk); @(negedge clk); s_axi_arvalid = 0;
        check_read_tlp(4'h6, 32'hB000_3000);
        @(negedge clk); tx_tlp_ready = 1; @(posedge clk); @(negedge clk); tx_tlp_ready = 0;

        send_completion(8'h06, 64'hAAAA_0000_0000_0001, 2'b00, 1'b0);
        wait (s_axi_rvalid);
        if (s_axi_rlast) fail("First beat of read burst incorrectly marked RLAST");
        if (s_axi_rdata !== 64'hAAAA_0000_0000_0001) fail("Read burst beat 0 data mismatch");
        else pass("Read burst beat 0 received correctly");
        @(negedge clk); s_axi_rready = 1; @(posedge clk); @(negedge clk); s_axi_rready = 0;

        check_read_tlp(4'h6, 32'hB000_3008);
        @(negedge clk); tx_tlp_ready = 1; @(posedge clk); @(negedge clk); tx_tlp_ready = 0;
        send_completion(8'h06, 64'hBBBB_0000_0000_0002, 2'b00, 1'b1);
        wait (s_axi_rvalid);
        if (!s_axi_rlast) fail("Last beat of read burst did not assert RLAST");
        if (s_axi_rdata !== 64'hBBBB_0000_0000_0002) fail("Read burst beat 1 data mismatch");
        else pass("Read burst final beat received correctly");
        @(negedge clk); s_axi_rready = 1; @(posedge clk); @(negedge clk); s_axi_rready = 0;

        $display("\n=== TEST 8: Completion Error -> AXI SLVERR ===");
        axi_read_single(4'h7, 32'hB000_4000);
        check_read_tlp(4'h7, 32'hB000_4000);
        @(negedge clk); tx_tlp_ready = 1; @(posedge clk); @(negedge clk); tx_tlp_ready = 0;
        send_completion(8'h07, 64'h0, 2'b01, 1'b1);
        wait (s_axi_rvalid);
        if (s_axi_rresp !== 2'b10) fail("PCIe error status was not mapped to AXI SLVERR");
        else pass("PCIe completion error mapped to AXI SLVERR");
        @(negedge clk); s_axi_rready = 1; @(posedge clk); @(negedge clk); s_axi_rready = 0;
```

```
388
389         $display("\n=== TEST 9: Wrong Completion Tag Rejection ===");
390         axi_read_single(4'h8, 32'hB000_5000);
391         check_read_tlp(4'h8, 32'hB000_5000);
392         @(negedge clk); tx_tlp_ready = 1; @(posedge clk); @(negedge clk); tx_tlp_ready = 0;
393
394         @(negedge clk);
395         rx_cpl_tag = 8'h09;
396         rx_cpl_data = 64'h9999_9999_9999_9999;
397         rx_cpl_status = 2'b00;
398         rx_cpl_eop = 1'b1;
399         rx_cpl_valid = 1'b1;
400         repeat (2) @(posedge clk);
401         if (rx_cpl_ready) fail("Bridge accepted a completion with the wrong tag");
402         if (s_axi_rvalid) fail("Wrong-tag completion reached AXI R channel");
403         else pass("Wrong completion tag correctly rejected");
404         @(negedge clk); rx_cpl_valid = 0;
405
406         send_completion(8'h08, 64'h8888_7777_6666_5555, 2'b00, 1'b1);
407         wait (s_axi_rvalid);
408         if (s_axi_rdata !== 64'h8888_7777_6666_5555) fail("Correct completion was not accepted");
409         else pass("Correct completion accepted after wrong-tag attempt");
410         @(negedge clk); s_axi_rready = 1; @(posedge clk); @(negedge clk); s_axi_rready = 0;
411
412         $display("\n============================================================");
413         $display("Stage-1 Verification Summary");
414         $display("PASS checks : %0d", pass_count);
415         $display("FAIL checks : %0d", errors);
416         if (errors == 0)
417             $display("RESULT    : ALL TESTS PASSED");
418         else
419             $display("RESULT    : TESTS FAILED");
420         $display("============================================================\n");
421
422         $finish;
423     end
424
425 endmodule
```